@invoice_auditor

Appears in: **DL-02**

orchestra/agents/invoice_auditor.py:18-111

(same parallel-agents sentence as above)

@invoice_auditor

orchestra/agents/invoice_auditor.py:18-111

```python
def run(brief: dict, llm=None) -> list:
    """Returns list of Finding dicts."""
    if llm is None:
        llm = get_llm()

    invoices = brief["invoices"]
    dupes    = _find_duplicates(invoices)
    findings = []

    for pair in dupes:
        result = call_json(
            llm,
            system=(
                "You are a financial auditor. Return ONLY valid JSON -- no markdown:\n"
                '{"risk_level": "HIGH|MEDIUM|LOW", '
                '"fraud_likelihood": "suspicious|likely_error|unknown", '
                '"confidence": 0.0-1.0, "reasoning": "..."}'
            ),
            user=(
                f"Duplicate invoice pair:\n"
                f"  Invoice A: {pair['invoice_1']}\n"
                f"  Invoice B: {pair['invoice_2']}\n"
                f"  Supplier: {pair['supplier_id']}\n"
                f"  Amount: ${pair['amount']}\n"
                f"  Days apart: {pair['days_apart']}\n\n"
                "Is this a billing error or potential fraud? Consider the time gap and amount."
            ),
        )

        if result.get("fallback"):
            risk            = "HIGH" if pair["days_apart"] <= 30 else "MEDIUM"
            fraud_likelihood = "unknown"
            confidence      = 1.0 if risk == "HIGH" else 0.7
            reasoning       = f"Rule-based: {pair['days_apart']} days apart"
        else:
            risk            = result.get("risk_level", "MEDIUM")
            fraud_likelihood = result.get("fraud_likelihood", "unknown")
            confidence      = float(result.get("confidence", 0.7))
            reasoning       = result.get("reasoning", "")

        findings.append({
            "agent": AGENT_NAME,
            "type": "duplicate_invoice",
            "severity": "warning",
            "sku_id": pair.get("sku_id"),
            "confidence": confidence,
            "data": {
                "invoice_1": pair["invoice_1"],
                "invoice_2": pair["invoice_2"],
                "supplier_id": pair["supplier_id"],
                "amount": pair["amount"],
                "days_apart": pair["days_apart"],
                "risk_level": risk,
                "fraud_likelihood": fraud_likelihood,
            },
            "reasoning": reasoning,
            "recommendation": "flag_duplicate",
        })

    return findings


def _find_duplicates(invoices: list) -> list:
    dupes = []
    seen  = set()
    for i, a in enumerate(invoices):
        for j, b in enumerate(invoices):
            if i >= j:
                continue
            key = tuple(sorted([a["invoice_id"], b["invoice_id"]]))
            if key in seen:
                continue
            if a["supplier_id"] != b["supplier_id"]:
                continue
            if abs(a["total_amount"] - b["total_amount"]) > 0.01:
                continue
            try:
                d1 = datetime.fromisoformat(a["date"])
                d2 = datetime.fromisoformat(b["date"])
            except (ValueError, KeyError):
                continue
            days = abs((d1 - d2).days)
            if days > 60:
                continue
            seen.add(key)
            dupes.append({
                "invoice_1": a["invoice_id"],
                "invoice_2": b["invoice_id"],
                "supplier_id": a["supplier_id"],
                "sku_id": a.get("sku_id"),
                "amount": a["total_amount"],
                "days_apart": days,
            })
    return dupes
```

What it shows: Specialist agent #3: finds duplicate invoice pairs (same supplier, same amount, ≤60 days apart) and LLM-classifies fraud vs billing error.

Source: orchestra/agents/invoice_auditor.py:18-111 · image rendered by build_snippets.py